LAPORAN UJIAN TENGAH SEMESTER GANJIL TA-2026/2027

Komputer Paralel (Sistem Terdistribusi) - Program Studi Teknik Informatika
Fakultas Teknik dan Ilmu Komputer, Universitas Cendekia Abditama

Konvolusi Citra Paralel (Gaussian Blur 5x5 dan Sobel): Perbandingan POSIX Threads dan OpenMP

[Nama] - NIM [NIM]
Dosen: Dr. Ir. Antonius Irianto. S, ST., MT | Opsi B | 2 November 2026

1. PENDAHULUAN

1.1 Latar Belakang

Konvolusi spasial adalah operasi dasar pengolahan citra: setiap piksel keluaran dihitung dari jumlah berbobot piksel-piksel tetangganya. Untuk citra W x H dengan kernel k x k, biaya komputasinya $O(W \cdot H \cdot k^2)$. Pada citra 2000x2000 dengan kernel Gaussian 5x5, ada 4 juta piksel dan sekitar 100 juta operasi perkalian-penjumlahan per pass, sehingga masalah ini cukup besar untuk diuntungkan paralelisme. Secara prinsip tiap piksel keluaran independen (data-parallel), tetapi implementasi yang benar menuntut perhatian pada tiga hal yang dituntut soal: (1) **piksel di area border**, yang kernel-nya memerlukan tetangga di luar citra; (2) **pembagian workload** yang membuat sebuah band memerlukan baris milik band tetangga (region overlap/halo); dan (3) perbandingan porsi **I/O** terhadap **komputasi**, karena bagian I/O bersifat serial dan membatasi percepatan menurut Hukum Amdahl.

Dua model pemrograman shared-memory dibandingkan di sini. **POSIX Threads (Pthreads)** menuntut programmer membuat, mempartisi pekerjaan, dan menyinkronkan thread secara eksplisit. **OpenMP** memindahkan sebagian besar pekerjaan itu ke compiler dan runtime melalui directive. Keduanya dijalankan pada inti komputasi yang identik agar perbedaan yang terukur benar-benar berasal dari mekanisme paralelisasinya.

1.2 Tujuan Implementasi

- Mengimplementasikan filter Gaussian blur 5x5 dan deteksi tepi Sobel pada citra grayscale PGM berukuran minimal 2000x2000 (diuji pada 2000x2000) tanpa library citra, dengan dua strategi border (zero-padding dan replicate).
- Membuat tiga versi: serial (baseline), Pthreads, dan OpenMP, serta membuktikan hasil paralel identik byte-per-byte dengan serial.
- Mengukur waktu eksekusi, speedup, dan efficiency untuk 1, 2, 4, dan 8 thread, memisahkan waktu I/O dan komputasi.
- Menganalisis secara kritis kompleksitas kode, bottleneck (Hukum Amdahl), anomali, dan trade-off kemudahan vs kontrol.

2. DESAIN IMPLEMENTASI

2.1 Algoritma Serial (Baseline)

Program membaca PGM (P5 biner atau P2 ASCII) dengan parser sendiri, menjalankan konvolusi, lalu menulis PGM P5. Satuan kerja terkecil adalah `filter_row(in, out, f, b, y)`: menghitung satu baris keluaran. Versi serial hanyalah loop `for y in 0..H-1` yang memanggil fungsi tersebut; kedua versi paralel memanggil fungsi yang sama sehingga inti komputasi tidak berbeda. Seluruh aritmetika memakai integer (Gaussian: bobot [1 4 6 4 1]⊗[1 4 6 4 1] dengan jumlah 256 lalu dibagi 256 dengan pembulatan; Sobel: Gx dan Gy integer, magnitudo $\sqrt{Gx^2+Gy^2}$ dibulatkan dan di-clamp ke 255), sehingga hasil paralel harus identik dengan serial.

```
for y in 0 .. H-1:                      # filter_row(y)
    if y berada di interior vertikal:   # y >= r dan y < H-r
        for x in 0 .. r-1:        out[y][x] = konvolusi_tepi(x, y)    # dengan pengecekan batas
        for x in r .. W-r-1:      out[y][x] = konvolusi_cepat(x, y)   # tanpa pengecekan batas
        for x in W-r .. W-1:      out[y][x] = konvolusi_tepi(x, y)
    else:                                    # baris paling atas/bawah: seluruhnya tepi
        for x in 0 .. W-1:        out[y][x] = konvolusi_tepi(x, y)
```

Penanganan border. Pada jalur tepi, piksel di luar citra diambil lewat `fetch()`: untuk *zero-padding* nilainya 0, untuk *replicate* koordinat di-clamp ke piksel tepi terdekat. Jalur cepat untuk piksel interior menghindari pengecekan batas pada setiap akses. Untuk Gaussian (r=2) hanya sekitar 0.40% piksel memakai jalur tepi, sehingga border tidak menjadi masalah performa, tetapi tetap harus benar. Karena citra disimpan row-major dan kernel menyapu baris demi baris, akses memori bersifat sekuensial dan ramah cache.

File	Peran
src/filter.c / filter.h	Inti konvolusi, border, versi serial
src/par_pthreads.c	Paralelisasi eksplisit dengan POSIX Threads
src/par_openmp.c	Paralelisasi dengan OpenMP (compiler directive)
src/pgm.c, src/main.c	I/O PGM tanpa library; driver CLI dan pengukur waktu
tools/gen_pgm.c, tools/pgmcmp.c	Pembangkit citra uji (seed tetap 42) dan pembanding byte-per-byte
tests/test_correct.c, scripts/bench.sh	Verifikasi correctness dan benchmark otomatis

2.2 Strategi Paralelisasi Pthreads

Pembagian workload. Domain dipartisi per *blok baris kontigu* (static). Dengan p thread dan H baris, thread i menerima ⌊H/p⌋ baris ditambah satu baris bila i < H mod p, sehingga selisih beban antar thread paling banyak satu baris. Pemilihan blok baris (bukan kolom atau tile) menjaga akses memori sekuensial dalam band dan hanya menyisakan dua batas band. Jumlah thread dibatasi tidak lebih dari H. Thread utama ikut mengerjakan band 0, sehingga hanya p-1 thread yang dibuat.

Region overlap (halo). Piksel di dekat batas band memerlukan baris tetangga dari band lain (r baris di atas/bawah). Karena citra masukan *read-only* selama komputasi dan tersimpan di memori bersama, setiap thread cukup membaca baris halo langsung dari citra masukan; tidak perlu menyalin region overlap. Ini pembeda penting dengan model memori terdistribusi (MPI), yang akan memerlukan pertukaran halo.

Mekanisme sinkronisasi. Tidak ada mutex: tiap thread menulis baris keluaran yang saling lepas sehingga tidak ada write-write conflict maupun race condition, dan masukan hanya dibaca. Satu-satunya sinkronisasi adalah `pthread_join` sebagai barrier di akhir. Bila `pthread_create` gagal, program menjalankan fallback serial agar hasil tetap benar.

```
Task tk[i] = { in, out, f, b, y0, y1 };           // partisi: y1-y0 = H/p (+1 bila i < H%p)
for (i = 1; i < p; i++) pthread_create(&th[i], NULL, worker, &tk[i]);
worker(&tk[0]);                                    // thread utama = band 0
for (i = 1; i < p; i++) pthread_join(th[i], NULL); // barrier
```

2.3 Strategi Paralelisasi OpenMP

Pemilihan pragma dan clause. Seluruh paralelisasi hanya satu directive pada loop baris terluar:

```
#pragma omp parallel for num_threads(nthreads) schedule(static) \
            default(none) shared(in, out, f, b, h)
for (int y = 0; y < h; y++)
    filter_row(in, out, f, b, y);
```

Elemen	Pilihan	Alasan
parallel for	loop baris (terluar)	Iterasi independen; granularitas besar (satu baris = ribuan piksel) sehingga overhead runtime kecil
schedule(static)	blok kontigu	Beban tiap baris seragam; lokalitas cache terbaik; setara dengan partisi manual Pthreads
num_threads(p)	p dari CLI	Memudahkan sweep 1, 2, 4, 8 tanpa mengubah OMP_NUM_THREADS
default(none)	scope eksplisit	Memaksa setiap variabel dideklarasikan, mencegah variabel shared tidak sengaja

Variabel	Scope	Alasan
in	shared	Citra masukan read-only; semua thread membaca (termasuk baris halo)
out	shared	Ditulis pada baris yang berbeda per iterasi, tidak ada konflik
f, b, h	shared	Konstanta selama region (jenis filter, border, tinggi)
y	private	Variabel loop; otomatis private pada worksharing loop

Tidak diperlukan clause `reduction` karena tidak ada akumulator bersama. Barrier implisit di akhir `parallel for` menggantikan `pthread_join`. Border dan halo ditangani dengan cara yang sama seperti Pthreads: baris halo dibaca langsung dari `in`.

3. HASIL PENGUJIAN

3.1 Lingkungan dan Metodologi

Item	Nilai
CPU	Intel(R) Xeon(R) Processor @ 2.80GHz
Core logis tersedia (nproc)	1
Compiler / OS	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0 / Linux 6.18.44-fc-v185
Flag kompilasi	gcc -O2 -Wall -Wextra -std=gnu11 -fopenmp (tanpa -ffast-math)
Data uji	PGM sintetis 2000x2000 (4 juta piksel, seed 42): gradien, lingkaran, persegi, noise
Pengukuran	1x warm-up lalu 20 repetisi; dilaporkan **waktu terbaik (min)** dan median; waktu baca/tulis diukur terpisah
Variasi	thread 1, 2, 4, 8 (Pthreads dan OpenMP); filter Gaussian dan Sobel; border replicate dan zero

Speedup dan efficiency dihitung dari waktu komputasi terbaik: $S(p) = T_{serial} / T_p$ dan $E(p) = S(p)/p$. Waktu terbaik dipilih karena lebih tahan terhadap gangguan sesaat dari sistem (scheduler, proses lain) pada benchmark CPU-bound; median tetap ditampilkan pada tabel. Catatan: file masukan sudah berada di page cache OS pada saat dibaca, sehingga waktu baca di sini lebih kecil daripada pembacaan dari disk dingin.

3.2 Verifikasi Correctness

Correctness diverifikasi dengan tiga lapis. **(1) Tes manual** pada citra konstan 5x5 bernilai 100 yang hasilnya dapat dihitung dengan tangan: Gaussian+replicate harus tetap 100 di semua piksel; Gaussian+zero pada sudut (0,0) hanya memiliki bobot 1D 6+4+1 = 11 sehingga (11·11·100+128)>>8 = 47; Sobel+replicate harus 0 (tidak ada tepi); Sobel+zero pada sudut menghasilkan Gx=Gy=300 sehingga magnitudo 424 di-clamp ke 255. **(2) Tes acak** membandingkan serial, Pthreads, dan OpenMP dengan implementasi referensi naif yang independen, pada enam ukuran citra (termasuk 1x7, 2x2, 5x5 yang lebih kecil dari kernel, dan 37x53 yang tidak habis dibagi jumlah thread), kedua filter, kedua border, dan 1 sampai 16 thread. Hasil: **534 pemeriksaan, 0 gagal -> SEMUA LULUS**. **(3) Perbandingan byte-per-byte** pada citra 2000x2000: seluruh 32 dari 32 konfigurasi paralel pada benchmark (identik dengan keluaran serial menurut `pgmcmp`).



Gambar 1. Potongan 400x400 dari tengah citra: masukan, hasil Gaussian 5x5, dan hasil Sobel (replicate).

3.3 Tabel Waktu Eksekusi

Tabel 1 dan 2 memuat waktu (ms) untuk border *replicate*. Kolom Total = baca + komputasi (min) + tulis. Speedup dihitung terhadap komputasi serial.

Tabel 1. Gaussian 5x5, border replicate (ms)

Mode	p	Baca	Komp. median	Komp. min	Tulis	Total	Speedup	Efficiency
Serial	1	4.16	51.78	50.00	1.10	55.26	1.00	1.00
Pthreads	1	1.63	86.87	50.35	1.39	53.36	0.99	0.99
Pthreads	2	1.64	50.17	49.09	1.30	52.04	1.02	0.51
Pthreads	4	1.55	49.97	49.03	2.30	52.87	1.02	0.25
Pthreads	8	1.65	53.42	49.49	2.69	53.84	1.01	0.13
OpenMP	1	1.97	53.38	49.58	2.94	54.49	1.01	1.01
OpenMP	2	2.22	54.00	49.18	2.75	54.14	1.02	0.51
OpenMP	4	2.18	53.75	50.83	2.83	55.84	0.98	0.25
OpenMP	8	1.53	51.00	50.15	2.25	53.94	1.00	0.12

Tabel 2. Sobel 3x3, border replicate (ms)

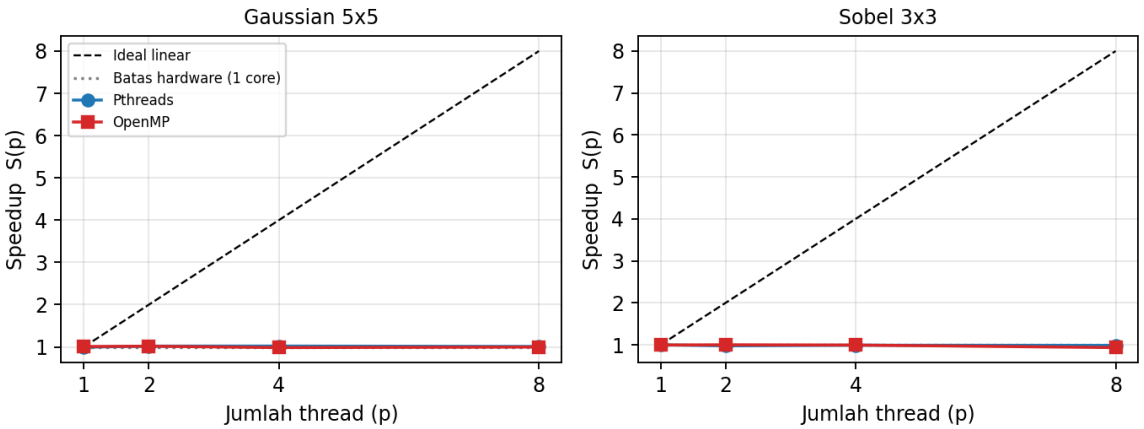
Mode	p	Baca	Komp. median	Komp. min	Tulis	Total	Speedup	Efficiency
Serial	1	1.82	60.10	41.45	3.79	47.06	1.00	1.00
Pthreads	1	2.18	42.15	41.50	4.30	47.98	1.00	1.00
Pthreads	2	1.51	81.25	42.46	2.31	46.29	0.98	0.49
Pthreads	4	1.50	43.47	41.86	2.80	46.15	0.99	0.25
Pthreads	8	2.06	44.78	42.03	2.82	46.91	0.99	0.12
OpenMP	1	1.63	44.35	41.67	5.39	48.69	0.99	0.99
OpenMP	2	2.14	42.42	41.44	2.61	46.19	1.00	0.50
OpenMP	4	1.52	51.95	41.68	2.67	45.88	0.99	0.25
OpenMP	8	1.67	75.77	44.37	2.90	48.94	0.93	0.12

Tabel 3. Pengaruh strategi border pada komputasi serial (ms, min)

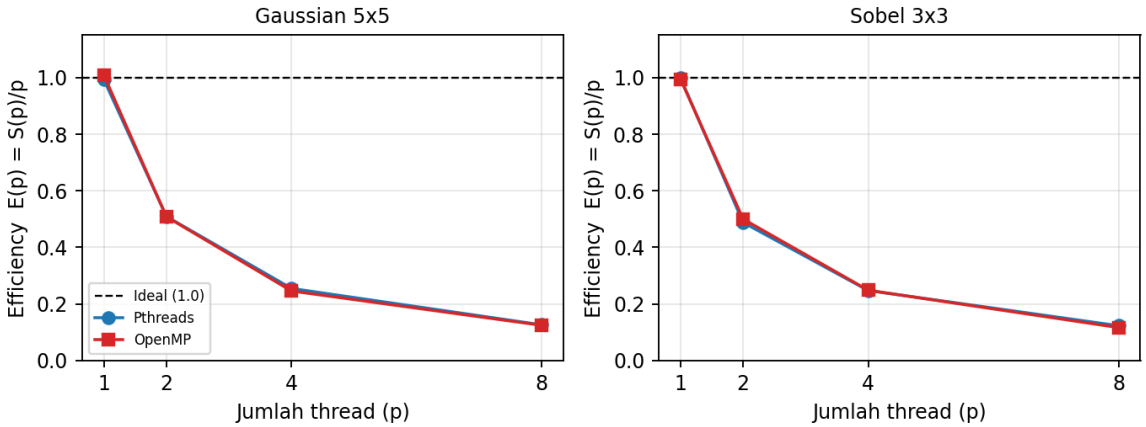
Filter	Replicate	Zero-padding	Selisih
Gaussian 5x5	50.00	49.35	-1.3 %
Sobel 3x3	41.45	42.44	2.4 %

Selisih antara dua strategi border sangat kecil karena hanya piksel di tepi yang berbeda jalurnya; strategi border mempengaruhi *nilai* piksel tepi (mis. Sobel+zero menghasilkan garis tepi palsu di pinggir citra), bukan performa.

3.4 Grafik Speedup dan Efficiency



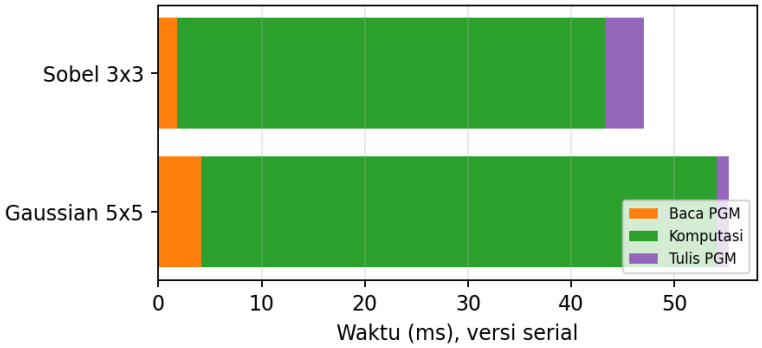
Gambar 2. Speedup waktu komputasi terhadap jumlah thread (border replicate). Garis titik abu-abu = batas hardware min(p, core).



Gambar 3. Efficiency E(p) = S(p)/p.

Pada 8 thread, speedup komputasi Gaussian adalah **1.01x** (Pthreads) dan **1.00x** (OpenMP); untuk Sobel **0.99x** dan **0.93x**. Mesin uji memiliki **1** core logis, sehingga batas atas speedup adalah min(8, 1) = 1x. Selisih rata-rata waktu komputasi antara Pthreads dan OpenMP pada semua titik uji adalah 1.9%.

3.5 Analisis I/O-bound vs Compute-bound



Gambar 4. Dekomposisi waktu eksekusi serial: baca, komputasi, tulis.

Pada versi serial, I/O (baca + tulis) hanya 9.5% dari total untuk Gaussian dan 11.9% untuk Sobel. Waktu komputasi sekitar 12.5 ns/piksel (Gaussian) dan 10.4 ns/piksel (Sobel), sedangkan trafik data efektif hanya sekitar 160 MB/s - jauh di bawah bandwidth memori mesin modern (orde puluhan GB/s). Artinya inti konvolusi bersifat **compute-bound**: tiap byte yang dibaca dipakai kembali oleh 25 (Gaussian) atau 9 (Sobel) operasi kali-tambah dari cache, sehingga menambah core akan meningkatkan throughput, bukan membentur dinding bandwidth. Sebaliknya, **end-to-end** program menjadi lebih dekat ke I/O-bound seiring bertambahnya thread, karena bagian I/O tetap serial (dibahas pada 4.2).

4. ANALISIS KRITIS

4.1 Perbandingan Kompleksitas Kode (LOC dan Maintenance)

File	Baris total	LOC kode (tanpa komentar/kosong)
filter.c (inti + serial, dipakai bersama)	124	94
par_pthreads.c	67	45
par_openmp.c	22	9
pgm.c	77	64
main.c	107	87

Tabel 4. Ukuran kode (dihitung otomatis oleh make_report.py)

Lapisan paralel Pthreads membutuhkan **45 LOC** kode, sedangkan OpenMP **9 LOC** (rasio 5.0x), padahal inti komputasinya sama. Selisih itu bukan sekadar jumlah baris, melainkan *tanggung jawab* yang dipindahkan ke programmer: mendefinisikan struct argumen, fungsi worker dengan signature `void*(void*)`, menghitung partisi dengan penanganan sisa (H mod p), alokasi array thread/task, memeriksa kegagalan `pthread_create`, dan join. Setiap bagian itu adalah tempat potensi bug (off-by-one pada band, kebocoran memori, lupa join). Pada OpenMP, satu pragma menggantikan seluruhnya dan versi serial tetap valid bila compiler tidak mendukung OpenMP (pragma diabaikan), sehingga *incremental parallelization* aman. Dari sisi maintenance, perubahan kebijakan penjadwalan di OpenMP cukup mengganti clause (`schedule(dynamic, 16)`), sedangkan di Pthreads harus menulis ulang partisi atau menambah antrean tugas dengan mutex/atomic.

4.2 Analisis Bottleneck dan Hukum Amdahl

Hukum Amdahl: S(p) = 1 / (f + (1-f)/p), dengan f fraksi serial. Pada program ini bagian serialnya adalah baca PGM, alokasi, dan tulis PGM; inti konvolusi paralel. Dari pengukuran serial: f = 0.095 (Gaussian) dan f = 0.119 (Sobel). Batas Amdahl pada p→∞ adalah 1/f = 10.5x untuk Gaussian dan 8.4x untuk Sobel - artinya semakin ringan komputasinya (Sobel), semakin cepat I/O menjadi pembatas. Tabel 5 membandingkan batas Amdahl dengan speedup end-to-end terukur.

Filter	p	Batas hardware min(p,core)	Amdahl (f dari serial)	Terukur Pthreads	Terukur OpenMP
Gaussian 5x5	1	1.00	1.00	1.04	1.01
Gaussian 5x5	2	1.00	1.83	1.06	1.02
Gaussian 5x5	4	1.00	3.11	1.05	0.99
Gaussian 5x5	8	1.00	4.80	1.03	1.02
Sobel 3x3	1	1.00	1.00	0.98	0.97
Sobel 3x3	2	1.00	1.79	1.02	1.02
Sobel 3x3	4	1.00	2.95	1.02	1.03
Sobel 3x3	8	1.00	4.36	1.00	0.96

Tabel 5. Speedup end-to-end (baca+komputasi+tulis) vs batas Amdahl. Amdahl dihitung mengasumsikan p core tersedia.

Selain Amdahl, **Karp-Flatt metric** e = (1/S - 1/p) / (1 - 1/p) mengukur fraksi serial *efektif* dari data terukur. Nilai e yang bertambah seiring p menandakan overhead paralel (pembuatan thread, kontensi memori, ketidakseimbangan beban) dan bukan sekadar bagian serial tetap.

Filter	Mode	e (p=2)	e (p=4)	e (p=8)
Gaussian 5x5	Pthreads	0.964	0.974	0.989
Gaussian 5x5	OpenMP	0.967	1.022	1.004
Sobel 3x3	Pthreads	1.049	1.013	1.016
Sobel 3x3	OpenMP	1.000	1.008	1.081

Tabel 6. Karp-Flatt metric dari speedup komputasi.

4.3 Pembahasan Anomali

Anomali utama: speedup tidak mengikuti garis ideal. Mesin uji hanya menyediakan 1 core logis, sehingga thread melebihi 1 tidak dapat berjalan serentak; mereka hanya di-time-slice oleh scheduler OS. Batas speedup nyata adalah min(p, 1) = 1x, dan hasil terukur (1.01x - 1.00x pada 8 thread untuk Gaussian) konsisten dengan batas itu. Ini **bukan** cacat implementasi: data justru menunjukkan *overhead* paralelisme sangat kecil, yaitu perubahan waktu komputasi pada 8 thread dibanding serial hanya -1.0% / +1.4% (Pthreads, Gaussian/Sobel) dan +0.3% / +7.0% (OpenMP). Kurva speedup yang bermakna hanya dapat diperoleh pada mesin dengan ≥8 core fisik (jalankan `make bench && make report` di mesin tersebut).

Anomali kedua: noise pengukuran. 10 dari 36 konfigurasi memiliki median lebih dari 30% di atas waktu terbaik (terburuk: Sobel 3x3, border zero, openmp, p=4: median 84.54 ms vs min 42.99 ms). Pola seperti ini khas lingkungan yang berbagi CPU (VM/container, proses lain, scheduler). Itulah alasan laporan memakai waktu terbaik dari beberapa repetisi untuk speedup, dan menampilkan

median sebagai pembanding. Untuk klaim yang lebih kuat sebaiknya dilaporkan simpangan baku dari banyak repetisi pada mesin yang idle.

Anomali ketiga (non-masalah): Pthreads vs OpenMP. Selisih rata-rata waktu komputasi hanya 1.9%. Karena keduanya memanggil `filter_row` yang sama dengan partisi blok-baris yang setara, perbedaan kinerja yang berarti memang tidak diharapkan; yang membedakan keduanya adalah biaya *pengelolaan* thread dan kemampuan menyetel penjadwalan, bukan kecepatan inti komputasi.

4.4 Trade-off antara Kemudahan vs Kontrol

Aspek	Pthreads	OpenMP
Kemudahan menulis	Rendah: 45 LOC, banyak boilerplate	Tinggi: 9 LOC, satu pragma
Kontrol partisi/penjadwalan	Penuh: pola partisi, antrean tugas, afinitas, prioritas dapat dibuat sendiri	Terbatas pada schedule(static/dynamic/guided), collapse, proc_bind
Sinkronisasi	Manual: mutex, condvar, barrier, atomic	Directive: barrier, critical, atomic, reduction
Penanganan error	Eksplisit (kegagalan pthread_create dapat ditangani)	Hampir tidak ada; gagal dibuat thread ditangani runtime
Risiko bug	Lebih tinggi (partisi, memori, join), data race tetap mungkin di keduanya	Lebih rendah, namun scope salah (shared/private) menghasilkan race yang sulit dilacak
Portabilitas	POSIX (Linux/macOS); Windows perlu lapisan kompatibilitas	Lintas platform bila compiler mendukung OpenMP
Pemeliharaan	Kode paralel terjalin dengan logika aplikasi	Kode serial tetap terbaca; paralelisasi bersifat anotasi
Cocok untuk	Pipeline/producer-consumer, thread berumur panjang, kontrol real-time	Loop data-parallel numerik seperti konvolusi ini

Kelemahan kedua pendekatan yang perlu dicatat: (1) false sharing pada baris keluaran di batas band dapat terjadi, tetapi dampaknya diabaikan di sini karena hanya p-1 batas dibanding 2000 baris; (2) partisi statis bergantung pada asumsi beban per baris seragam - benar untuk konvolusi, tetapi tidak untuk filter adaptif, yang memerlukan dynamic scheduling atau antrean tugas; (3) penghitungan *waktu paralel* hanya mencakup region komputasi, sehingga manfaat nyata bagi pengguna dibatasi oleh Hukum Amdahl (Bagian 4.2).

Keterbatasan pengujian. Hasil berasal dari satu mesin/lingkungan; citra masukan sintetis dan berada di page cache OS; satu ukuran citra; simpangan baku tidak dilaporkan. Optimasi lanjutan yang tidak diimplementasikan: Gaussian separable (dua pass 1D, 10 vs 25 perkalian per piksel), vektorisasi SIMD eksplisit, dan tiling untuk cache.

5. KESIMPULAN

5.1 Rekomendasi Penggunaan Pthreads vs OpenMP

- **Gunakan OpenMP** untuk loop data-parallel seperti konvolusi: 9 LOC vs 45 LOC, kode serial tetap utuh, dan kinerjanya setara (selisih rata-rata 1.9% pada pengukuran ini).
- **Gunakan Pthreads** bila memerlukan kontrol yang tidak dapat diekspresikan dengan directive: thread berumur panjang, pipeline baca-proses-tulis (untuk menumpuk I/O dengan komputasi), antrean tugas kustom, afinitas/prioritas, atau penanganan kegagalan pembuatan thread.
- Dalam program ini, peningkatan end-to-end berikutnya paling efektif datang dari **mengurangi bagian serial I/O** (f = 0.095-0.119) dan memperkecil biaya per piksel (Gaussian separable, SIMD), bukan dari menambah thread semata.

5.2 Pembelajaran yang Didapat

- Menghilangkan dependency antar-piksel di border tidak memerlukan sinkronisasi pada shared memory bila masukan read-only dan keluaran dipartisi tanpa tumpang tindih; kesalahan border lebih mungkin muncul pada logika indeks, itulah sebabnya tes dibuat pada citra yang lebih kecil dari kernel dan ukuran yang tidak habis dibagi jumlah thread.
- Jumlah thread tidak boleh melebihi core fisik jika tujuannya speedup: pengukuran harus selalu dibaca bersama spesifikasi mesin (batas min(p, core)).

- Tanpa memisahkan waktu I/O dan komputasi, hasil end-to-end akan menyesatkan; Hukum Amdahl terlihat langsung pada data.
- Metodologi benchmark penting: warm-up, banyak repetisi, waktu terbaik vs median, dan verifikasi correctness otomatis pada setiap konfigurasi.

6. DAFTAR PUSTAKA

[1] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings* (Spring Joint Computer Conference), 30, 483-485.

[2] Butenhof, D. R. (1997). *Programming with POSIX Threads*. Addison-Wesley.

[3] Chapman, B., Jost, G., & van der Pas, R. (2008). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.

[4] Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.

[5] Hennessy, J. L., & Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.

[6] Karp, A. H., & Flatt, H. P. (1990). Measuring parallel processor performance. *Communications of the ACM*, 33(5), 539-543.

[7] OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*. https://www.openmp.org

[8] Poskanzer, J. *PGM Format Specification*. Netpbm. https://netpbm.sourceforge.net/doc/pgm.html

[9] Sobel, I., & Feldman, G. (1968). A 3x3 isotropic gradient operator for image processing. Presentasi pada Stanford Artificial Intelligence Project (SAIL).